

BÁO CÁO PHÂN TÍCH PHỐI HỢP CPU-RAM-STORAGE & XỬ LÝ LỖI ĐƯỜNG DẪN

Dự án: Ứng dụng Shopee-Lite Python

Ngày: 05/10/2026

I. Giải thích lỗi File Not Found - Tại sao đường dẫn tuyệt đối gây lỗi khi đổi máy?

1. Định nghĩa

Đường dẫn tuyệt đối: Viết đầy đủ vị trí từ gốc hệ thống tệp, ví dụ:

python

Run

Máy bạn

"D:/MyProject/assets/logo.png"

Hoặc Linux/macOS

"/home/nguoidung/Project/assets/logo.png"

Đường dẫn tương đối: Xác định vị trí tệp dựa trên thư mục chứa mã nguồn, không phụ thuộc vào cấu trúc ổ đĩa hoặc tên người dùng.

2. Nguyên nhân lỗi khi đổi máy

Table

Yếu tố Ảnh hưởng

Khác tên ổ đĩa Máy bạn dùng D:/, máy bạn thân có thể chỉ có C:/ hoặc E:/ -> đường dẫn không tồn tại

Khác tên thư mục gốc Bạn đặt trong MyProject/, bạn thân giải nén vào Shopee Lite/ -> sai đường dẫn

Khác hệ điều hành Windows dùng dấu gạch chéo ngược \, Linux/macOS dùng / -> chuỗi đường dẫn không hợp lệ

Khác tên người dùng Đường dẫn chứa tên tài khoản người dùng -> mỗi máy khác nhau hoàn toàn

=> Kết luận: Đường dẫn tuyệt đối "cứng" gắn với một máy cụ thể. Khi chuyển dự án, vị trí thay đổi -> Python không tìm thấy tệp -> FileNotFoundError.

II. Phân tích luồng IPO: Storage => RAM => CPU

1. Mô hình IPO áp dụng cho việc nạp tài nguyên

Table

Giai đoạn Hoạt động chi tiết Vai trò thiết bị

Input - Đọc từ Storage Ứng dụng yêu cầu mở file ảnh/logo => Hệ điều hành truy cập ổ đĩa (SSD/HDD), tìm cụm dữ liệu, đọc byte thô Storage: Nơi lưu trữ vĩnh viễn, tốc độ chậm nhất (0,5–3 GB/s SSD, chậm hơn RAM hàng trăm lần)

Process - Nạp vào RAM Dữ liệu thô được chuyển vào bộ nhớ chính => Giải mã định dạng ảnh -> Chuyển thành mảng điểm ảnh/dữ liệu mà thư viện đồ họa có thể hiển thị

RAM: Bộ nhớ làm việc tạm, tốc độ trung gian (~20-80 GB/s), dữ liệu mất khi tắt máy

Output - CPU xử lý & hiển thị CPU thực thi lệnh vẽ ảnh lên giao diện, tính toán bố cục, cập nhật màn hình CPU: Bộ xử lý, thực thi logic, tốc độ nhanh nhất, chờ đợi nếu dữ liệu chưa vào RAM

2. Tại sao máy bạn thân bị treo - RAM 98%?

Khi đường dẫn sai => ứng dụng liên tục thử lại đọc file không tồn tại => tạo vòng lặp vô hạn hoặc mở nhiều luồng/tiến trình chờ tài nguyên

Mỗi lần thử lại cấp phát thêm bộ nhớ mà không giải phóng => Rò rỉ bộ nhớ => RAM đầy dần

Khi RAM đạt gần 100%, hệ điều hành dùng tệp hoán đổi (swap) trên ổ đĩa => tốc độ giảm cực mạnh => máy bị đóng băng, phản hồi chậm

CPU bị chiếm dụng xử lý lặp lại yêu cầu thất bại => tải cao nhưng không hoàn thành công việc hữu ích

III. Giải pháp kỹ thuật

1. Sử dụng đường dẫn tương đối - Ký hiệu . và ..

Table

Ký hiệu Ý nghĩa

. Thư mục hiện tại (chứa file .py đang chạy)

.. Lùi về thư mục cha

Cấu trúc thư mục đề xuất

plaintext

Shopee-Lite/

main.py

assets/

logo.png

banner.jpg

utils/

 helper.py

Cách viết đúng - Sử dụng os.path hoặc pathlib (độc lập HĐH)

Cách 1 - Dùng os.path (dễ hiểu, tương thích rộng):

python

Run

import os

```
# Thư mục chứa file mã nguồn hiện tại
```

```
thu_muc_goc = os.path.dirname(__file__)
```

```
# Đường dẫn tương đối đến tài nguyên
```

```
duong_dan_logo = os.path.join(thu_muc_goc, "assets", "logo.png")
```

```
# Sử dụng
```

```
if os.path.exists(duong_dan_logo):
```

```
    logo = load_image(duong_dan_logo)
```

```
else:
```

```
    print("Không tìm thấy logo tại:", duong_dan_logo)
```

Cách 2 — Dùng pathlib (hiện đại, rõ ràng):

python

Run

```
from pathlib import Path
```

```
thu_muc_goc = Path(__file__).resolve().parent
```

```
duong_dan_logo = thu_muc_goc / "assets" / "logo.png"
```

```
if duong_dan_logo.exists():
```

```
    logo = load_image(duong_dan_logo)
```

Giải thích ký hiệu trong thực tế:

"./assets/logo.png" = thư mục hiện tại => vào assets => lấy logo.png

"../assets/logo.png" = lùi ra thư mục cha => vào assets => lấy logo.png (dùng khi code ở thư mục con như utils/)

2. 3 bước giảm lag => Dựa trên Task Manager

Table

Bước Hành động cụ thể Tác dụng

Bước 1 - Ngăn chặn lặp vô hạn Thêm kiểm tra tồn tại file trước khi nạp, chỉ thử 1 lần, báo lỗi rõ ràng nếu thiếu => không tạo vòng lặp cấp phát bộ nhớ Giảm ngay tải CPU, ngăn rò rỉ RAM

Bước 2 - Giải phóng bộ nhớ không dùng Đóng đối tượng ảnh không hiển thị, xóa biến không cần thiết bằng del, gọi gc.collect(); giới hạn số ảnh nạp cùng lúc. Giảm mức sử dụng RAM xuống mức bình thường (-30-50%)

Bước 3 - Tối ưu nạp tài nguyên Nạp ảnh khi cần hiển thị thay vì nạp tất cả lúc khởi động; giảm kích thước ảnh gốc; dùng định dạng nhẹ (.webp thay vì .png lớn) Giảm áp lực cả Storage (đọc nhanh hơn) + RAM (dữ liệu nhỏ hơn) + CPU (giải mã nhanh hơn)